

Practical Work 4: MapReduce Word Count (C Implementation)

Tran Tuan Vu -23BI14459

December 11, 2025

1 Implementation Choice

We implemented a custom MapReduce framework using **C** and **POSIX Threads (pthread)**.

- **Why C?** We chose C to gain a deeper understanding of low-level memory management and thread synchronization without relying on high-level abstractions like C++ containers.
- **Why pthreads?** The POSIX thread library is the standard for multithreading in UNIX-like systems, allowing us to simulate parallel "Worker" nodes efficiently on a single multi-core machine.

2 System Design

Our design manually implements the MapReduce phases:

1. **Input Splitting:** The master reads the file into a large buffer and splits it into two segments (chunks) using pointer arithmetic.
2. **Map Phase (Parallel):** Two threads execute the `mapper` function. Each mapper parses its chunk and fills a local array of `KeyValue` structs (emitting `<word, 1>`).
3. **Shuffle & Sort:** The master thread collects arrays from all mappers into a single large array and uses `qsort` to sort them alphabetically. This groups identical words together (e.g., `apple, apple, banana`).
4. **Reduce Phase:** The `reducer` function iterates linearly through the sorted array, summing consecutive counts for the same word and printing the result.

3 Implementation Details

3.1 Data Structures

We defined a simple struct to hold the data, simulating the key-value pair used in distributed systems.

```
1 typedef struct {  
2     char word[50];  
3     int count;  
4 } KeyValue;
```

Listing 1: Key-Value Struct

3.2 The Mapper

The mapper runs in a separate thread. It tokenizes strings and stores them in a dynamically allocated results array.

```
1 void *mapper(void *arg) {
2     // ... Parsing logic ...
3     if (isalpha(ptr[i])) {
4         // Build word character by character
5     } else {
6         // Emit (Word, 1)
7         strcpy(args->results[count].word, word);
8         args->results[count].count = 1;
9     }
10    return NULL;
11 }
```

Listing 2: Mapper Logic

3.3 The Shuffle (Sorting)

We rely on the standard C library's `qsort` to bring identical keys together, which is a prerequisite for the reducer.

```
1 int compare_kv(const void *a, const void *b) {
2     return strcmp(((KeyValue *)a)->word, ((KeyValue *)b)->word);
3 }
4 // Called in main:
5 qsort(all_data, total_items, sizeof(KeyValue), compare_kv);
```

Listing 3: Comparison Function

4 Experimental Results

We executed the program on a text file containing the phrase: "hello world hello C programming world".

```
1 $ ./wordcount input.txt
2 [Master] Starting 2 Mapper threads (pthreads)...
3 ----- Final Word Counts -----
4 c: 1
5 hello: 2
6 programming: 1
7 world: 2
```

Execution Log